GitHub Guide



A beginner-friendly guide to GitHub, Markdown, collaboration, and building a professional developer profile.

Table of Contents

- History
- GitHub Accounts
  - Repositories
  - Licenses
  - The `.gitignore` File
  - The `README.md` File
- Markdown
- Writing Your Resume on GitHub

- GitHub Desktop
- GitHub Organizations
- GitHub Pages
- GitHub Issues
- Branches
- Pull Requests
- Merge
- Fork

History

To understand how a technology works and why it exists, we should first understand the problem it was designed to solve.

When Linus Torvalds was developing the Linux kernel, managing contributions and tracking changes across a large, distributed development team became increasingly challenging. In 2005, he created **Git**, a distributed version control system designed to make tracking changes, managing branches, and collaborating on software projects more efficient.

For a more detailed introduction to Git, check out the Git Guide.

GitHub was launched in 2008 as a platform built around Git. It made it easier for developers to host repositories online, contribute to open-source projects, review code, collaborate with teams, and showcase their work to potential employers.

In 2018, Microsoft acquired GitHub. This acquisition also strengthened the integration between GitHub and Microsoft's developer tools, including Visual Studio Code.

Today, GitHub is much more than a place to store code. It is a platform for collaboration, open-source development, project management, and professional networking.

GitHub Accounts

A GitHub account gives you access to repositories, collaboration tools, issue tracking, and other platform features.

Creating an Account

Follow these steps to create your account:

1. Visit GitHub and click **Sign up**.
2. Enter your email address and create a strong password.
3. Choose your username carefully because it becomes part of your public profile URL:

   `https://github.com/your-username`
4. Complete the verification process.
5. Open your profile at `https://github.com/your-username` and click **Edit profile**.
6. Add a profile picture, display name, bio, location, and other relevant information.
7. If applicable, include your university, organization, or personal website.

> **Tip:** Choose a professional username that is easy to remember and suitable for your future career.

A single email address can generally be associated with one GitHub account at a time. GitHub also supports changing the email address associated with an account.

Repositories

A **repository (Repo)** is a place where a project and its associated files are stored and tracked.

A repository can contain source code, documentation, images, configuration files, and other project resources. Git tracks changes to files, while GitHub hosts the repository online and provides collaboration tools.

Repositories can be public or private:

- **Public:** Anyone can view the repository.
- **Private:** Access is restricted to authorized users.

You can create a repository for a personal project, a university assignment, an open-source contribution, or your professional portfolio.

Creating a Repository

1. Sign in to GitHub.
2. Click the `+` icon and select **New repository**.
3. Enter a repository name and an optional description.
4. Choose whether the repository should be public or private.
5. Optionally, initialize it with a `README.md`, a `.gitignore` template, or a license.
6. Click **Create repository**.

Licenses

A license explains how other people are allowed to use, modify, distribute, and share your project's code.

Choosing a license is important, especially for open-source projects, because making a repository public does not automatically grant permission to reuse its contents.

Here are three common options:

MIT License

The MIT License is a permissive open-source license. It allows people to use, modify, distribute, and include your code in other projects, including commercial projects, provided they preserve the required copyright and license notices.

It is a popular choice for personal projects and open-source libraries.

Apache License 2.0

The Apache License 2.0 is another permissive open-source license. Like the MIT License, it allows broad reuse and redistribution.

It also includes explicit provisions related to patent rights and requires preservation of certain notices.

This can be a good choice for projects where clear patent terms are important.

No License

If you do not provide a license, your code is generally subject to the default copyright protections that apply to it.

Other people can view a public repository, but they do not automatically receive permission to copy, modify, distribute, or use its code.

> **Recommendation:** If you want others to use and contribute to your open-source project, choose an appropriate license. Read its terms before publishing.

For more information, visit Choose a License.

The `.gitignore` File

The `.gitignore` file tells Git which untracked files and directories it should ignore.

During development, many files are generated automatically or contain information that should not be included in the repository. Examples include temporary files, build outputs, local

configuration files, and dependency directories.

A `.gitignore` file helps prevent these files from being accidentally added to version control.

Example

```
# Dependencies
node_modules/

# Build outputs
dist/
build/

# Environment variables and local secrets
.env
.env.local

# Python cache
__pycache__/
*.py[cod]

# Virtual environments
.venv/
venv/

# Operating system files
.DS_Store
Thumbs.db

# Editor settings
.vscode/
```

Common `.gitignore` Patterns

Pattern	Meaning
`file.txt`	Ignore a file matching this path or name
`*.log`	Ignore files ending in `.log`
`temp/`	Ignore a directory named `temp`
`!important.log`	Exclude a matching file from the ignore rules
`/config.json`	Ignore `config.json` at the repository root

Pattern	Meaning
`**/cache/`	Ignore directories named `cache` at any depth

Important Notes

- Git ignores matching files that are not already tracked.
- Adding a file to `.gitignore` does not automatically remove it from Git if it has already been committed.
- Never commit passwords, API keys, access tokens, or other secrets.
- If a sensitive file has already been pushed, simply ignoring it is not enough. Remove the exposed secret from the relevant service and rotate or revoke it immediately.

You can find useful templates in GitHub's gitignore repository.

The `README.md` File

A `README.md` file introduces your repository and explains what the project does.

It is usually displayed automatically on the repository's main page, making it one of the first things visitors see.

A good README may include:

- Project title and description
- Features
- Screenshots or demonstrations
- Installation instructions
- Usage examples
- Technologies used
- Project structure
- Contribution guidelines
- License information
- Contact details

The `.md` extension stands for Markdown, a lightweight markup language used to format text.

A clear README helps other developers understand your project quickly and makes your repository more professional.

Markdown

Markdown is a lightweight markup language created by John Gruber, with contributions from Aaron Swartz, and first released in 2004.

Its goal is to make writing formatted text simple and readable without requiring lengthy HTML tags. Markdown can be converted into HTML and is widely used for documentation, README files, issues, pull requests, and technical writing.

GitHub supports its own extended version of Markdown, commonly known as **GitHub Flavored Markdown (GFM)**.

Basic Markdown Syntax

1. Headings

Use `#` to create headings. The number of `#` symbols determines the heading level.

```
# Heading 1
## Heading 2
### Heading 3
#### Heading 4
##### Heading 5
###### Heading 6
```

Use headings in a logical order to organize your documentation.

2. Text Formatting

Bold text:

```
**This text is bold**
__This text is also bold__
```

Italic text:

```
*This text is italic*
_This text is also italic_
```

Bold and italic text:

```
***This text is bold and italic***
```

~~Strikethrough text:~~

```
~~This text is crossed out~~
```

3. Links

Use the following syntax to create a hyperlink:

```
[GitHub](https://github.com/)
```

You can also link to a specific section of the same document:

```
[Go to History](#history)
```

4. Images

Use an exclamation mark followed by the alternative text and the image URL:

```
![GitHub Logo](https://github.githubassets.com/images/modules/logos_page/GitHub-Mark.png)
```

The alternative text describes the image if it cannot be displayed and improves accessibility.

You can also link an image to another page:

```
[![GitHub Logo](https://github.githubassets.com/images/modules/logos_page/GitHub-Mark.png)](https
```

5. Lists

Unordered Lists

Use `-`, `*`, or `+` to create unordered lists.

```
- First item
- Second item
- Third item
```

Ordered Lists

Use numbers followed by periods:

```
1. First item
2. Second item
3. Third item
```

Nested Lists

Indent nested items beneath their parent items:

```
- Programming Languages
  - Python
  - JavaScript
  - C++
- Tools
  - Git
  - GitHub
```

6. Horizontal Rules

Use three or more hyphens, asterisks, or underscores to create a horizontal line.

```
---
```

This is useful for separating major sections of a document.

7. Blockquotes

Use `>` to highlight quotations, notes, or important information.

```
> This is a blockquote.
>
> It can contain multiple paragraphs.
```

8. Inline Code and Code Blocks

Use single backticks for inline code:

```
Use `git status` to check your working tree.
```

Use three backticks to create a code block:

````
```python
print("Hello, World!")
```
````

You can specify a language after the opening backticks to enable syntax highlighting.

For example:

```javascript
console.log("Hello, World!");
```

9. Tables

Use pipes (`|`) to separate columns and hyphens (`-`) to define the header row.

```
| Name | Language | Level |
|---|---|---|
| Python | Programming | Intermediate |
| HTML | Markup | Beginner |
| CSS | Styling | Beginner |
```

You can also align columns:

```
| Left | Center | Right |
|:---|:---:|---:|
| A | B | C |
| D | E | F |
```

10. Task Lists

GitHub supports interactive-looking checklists in issues, pull requests, and Markdown documents.

```
- [x] Learn Git basics
- [x] Create a GitHub account
- [ ] Publish a project
- [ ] Contribute to an open-source repository
```

`[x]` represents a completed task, while `[ ]` represents an incomplete task.

11. Escaping Special Characters

If you want to display a Markdown character instead of using it as formatting syntax, place a backslash before it.

```
\*This text will not be italicized\*
```

This is useful when documenting Markdown syntax itself.

12. HTML in Markdown

GitHub allows certain HTML elements inside Markdown documents.

For example:

```
<p align="center">
  <strong>Welcome to my GitHub profile!</strong>
</p>
```

However, not every HTML element or attribute is supported. GitHub sanitizes rendered HTML for security, so some elements, attributes, and styling options may be removed or restricted.

Tip: Prefer standard Markdown whenever possible because it is easier to read and maintain.

For more information, visit the GitHub Flavored Markdown documentation.



Writing Your Resume on GitHub

GitHub allows you to create a personal profile README that appears directly on your profile page.

This is a useful way to introduce yourself, highlight your technical skills, showcase projects, and provide professional contact information.

Creating a Profile README

1. Create a new repository.
2. Name the repository exactly the same as your GitHub username.
3. Make sure the repository is public.
4. Add a `README.md` file.
5. Write your introduction, skills, projects, achievements, and contact information.
6. Commit your changes and visit your GitHub profile.

When the repository name matches your username, GitHub displays its README on your profile page.

For example, if your username is `example-user`, your repository should be named `example-user`.

Recommended Resume Structure

1. Introduction

Introduce yourself briefly.

Include your professional interests, current role or education, and the kind of work you enjoy.

2. Skills

Organize your skills into meaningful categories.

- **Programming Languages:** Python, JavaScript, C++, Java
- **Frameworks and Libraries:** React, Django, Express
- **Tools and Platforms:** Git, GitHub, Docker, Linux
- **Databases:** PostgreSQL, MySQL, MongoDB
- **Professional Skills:** Communication, teamwork, problem-solving

Only include technologies and skills you can reasonably demonstrate.

3. Certificates and Achievements

List relevant certificates, awards, courses, and other achievements.

Whenever possible, link directly to the certificate or its official verification page.

4. Portfolio and Projects

Showcase projects that demonstrate your abilities.

For each project, include:

- Project name and a short description
- Technologies used
- Main features
- Link to the source code
- Live demo, if available

Prioritize complete, useful projects over a long list of unfinished repositories.

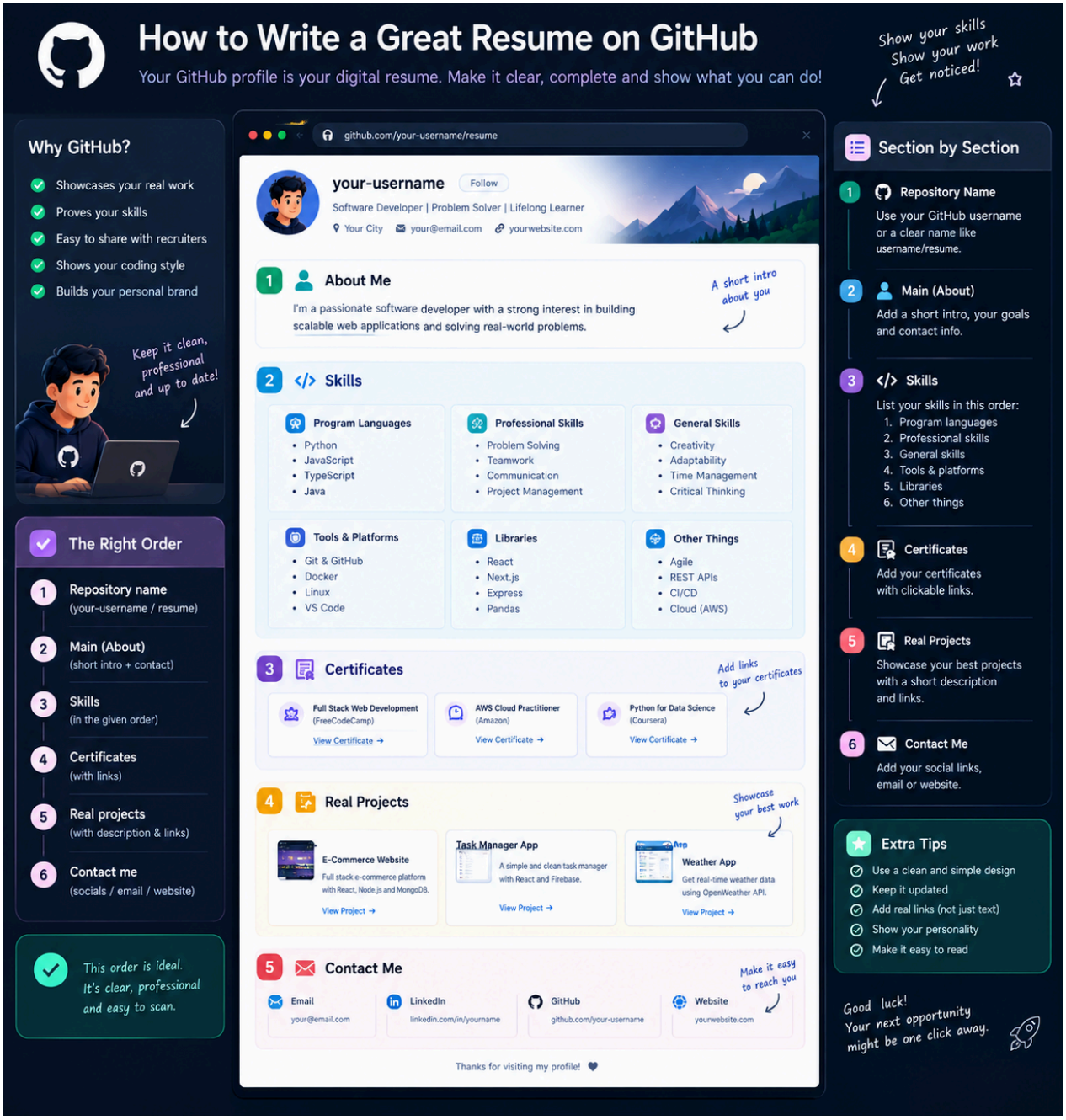
5. Contact Information

Add professional ways for people to contact you, such as:

- Email
- LinkedIn
- Personal website
- Portfolio
- Other relevant professional profiles

Avoid publishing sensitive personal information.

> **Tip:** Keep your profile README concise, organized, and up to date. Your strongest projects and most relevant skills should be easy to find.



GitHub Desktop

GitHub Desktop is a graphical application that makes it easier to work with Git repositories without relying entirely on command-line commands.

It provides a visual interface for reviewing changes, creating commits, managing branches, and synchronizing your local work with GitHub.

Installing GitHub Desktop

1. Download the application from the official website.

2. Install and open it.
3. Sign in with your GitHub account.
4. Clone an existing repository or create a new one.

Cloning a Repository

To download an existing GitHub repository to your computer:

1. Open GitHub Desktop.
2. Select **File → Clone Repository**.
3. Choose a repository from your GitHub account or enter its URL.
4. Select the local directory where you want to save it.
5. Click **Clone**.

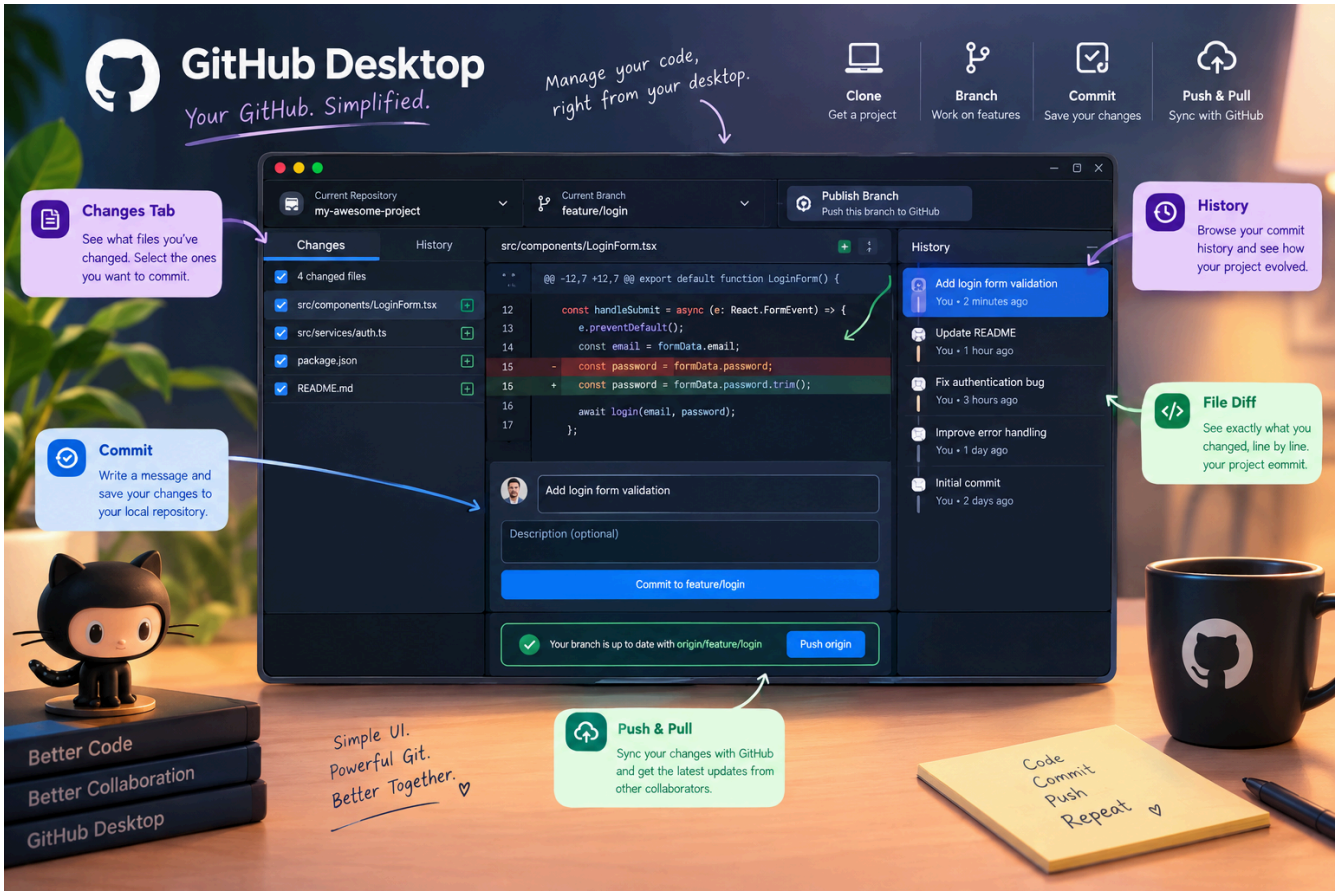
Cloning downloads the repository and its Git history to your computer, allowing you to work on it locally.

Making and Publishing Changes

After cloning a repository:

1. Open the repository's local folder in your preferred editor, such as Visual Studio Code.
2. Create, edit, or delete files as needed.
3. Return to GitHub Desktop to review the changes.
4. Enter a short, descriptive commit message.
5. Click **Commit to main** or commit to your current branch.
6. Click **Push origin** to upload your local commits to GitHub.

If you have not pushed your changes before, the application may show **Publish repository** or another publishing-related option instead.



Understanding the Main Terms

Clone

Downloads a repository from a remote location to your computer, including its Git history.

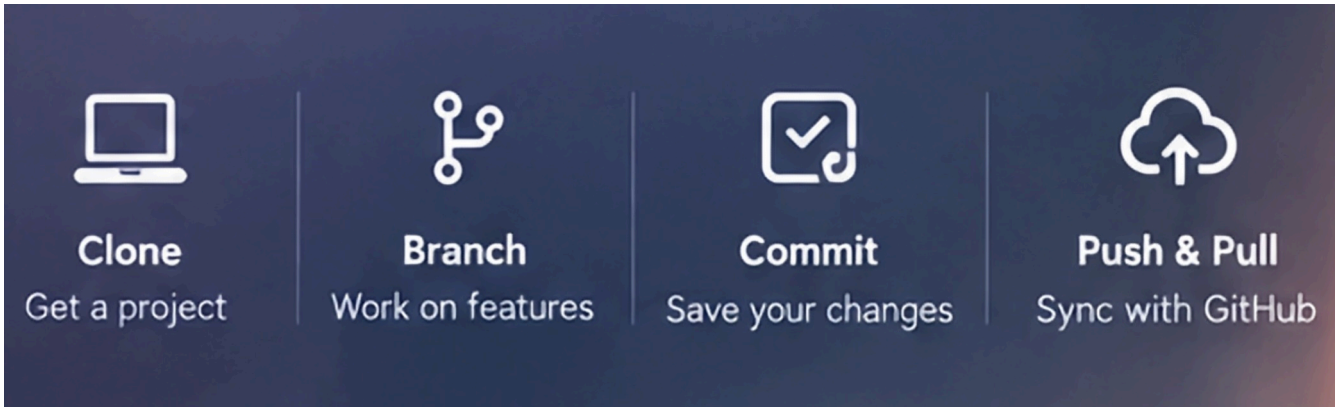
Commit

Records a snapshot of the staged changes in your local Git repository.

A commit is similar to saving a meaningful checkpoint in your project's history.

Push

Uploads your local commits to a remote repository, such as the one hosted on GitHub.



> **Important:** A commit does not automatically upload your changes to GitHub. You generally need to push your commits to publish them to the remote repository.

GitHub Organizations

A GitHub Organization is a shared account structure designed for companies, startups, educational groups, and development teams.

Organizations make it easier to manage repositories, control access, assign responsibilities, and collaborate across multiple projects.

Creating an Organization

1. Sign in to GitHub.
2. Open your account menu and navigate to the organization creation option in GitHub's settings.
3. Choose **New organization**.
4. Select an appropriate plan.
5. Enter the required organization details.
6. Invite team members and configure repository access.

You can start with a free plan when its available features meet your needs.

Organization Roles

Organization roles determine what members can do within the organization.

Common roles include:

- **Owner:** Has administrative control over the organization and its settings.

- **Member:** Has standard organization membership and access to resources according to assigned permissions.
- **Outside Collaborator:** Can be granted access to specific repositories without becoming an organization member.

Additional roles and permissions may be available depending on the organization's configuration and GitHub plan.

Always grant people only the permissions they need.

Creating an Organization Profile README

You can display a README on an organization's public profile.

Follow these steps:

1. Create a public repository named `.github` within the organization.
2. Inside that repository, create a directory named `profile`.
3. Add a `README.md` file inside the `profile` directory.
4. Write an introduction to the organization, its goals, projects, and contribution guidelines.

The resulting structure should look like this:

```
.github/
└── profile/
    └── README.md
```

GitHub can display this README on the organization's profile page.



GitHub Pages

GitHub Pages is a hosting service that allows you to publish static websites directly from a GitHub repository.

It is useful for personal websites, portfolios, project documentation, and landing pages.

GitHub Pages primarily supports static content such as HTML, CSS, and JavaScript. It does not directly run server-side applications such as a conventional PHP or Node.js backend.

Creating a GitHub Pages Website

1. Create a repository on GitHub.
2. Add your website files, including an `index.html` file.

3. Open the repository's **Settings**.
4. Select **Pages** from the sidebar.
5. Under the build and deployment settings, select the appropriate publishing source, such as a branch and folder, or configure GitHub Actions.
6. Save the settings and wait for the site to be published.

Understanding GitHub Pages URLs

There are two common types of GitHub Pages websites.

1. User or Organization Website

Create a repository named exactly:

```
your-username.github.io
```

The website will normally be available at:

```
https://your-username.github.io/
```

Replace `your-username` with your actual GitHub username.

2. Project Website

Create a repository with a regular name, such as:

```
my-project
```

The website will normally be available at:

```
https://your-username.github.io/my-project/
```

Replace the example username and repository name with your own.

> **Note:** The final URL depends on your GitHub account or organization, repository configuration, and publishing settings. Custom domains are also supported.

GitHub Pages is an excellent option for publishing a developer portfolio without managing your own web server.



GitHub Issues

GitHub Issues is a tool for tracking bugs, requesting features, asking questions, and organizing work related to a repository.

Although issues support discussion and comments, they are more than ordinary comments: each issue represents a task or topic that can be assigned, labeled, tracked, and discussed.

Common Uses

- **Bug Reports:** Describe unexpected behavior or errors.
- **Feature Requests:** Suggest new functionality.
- **Questions:** Ask questions about the project.

- **Task Tracking:** Organize planned work and improvements.

Writing a Good Issue

A useful issue should be clear, specific, and easy to reproduce when reporting a bug.

For a bug report, include:

1. **Title:** A short description of the problem.
2. **Description:** Explain what happened.
3. **Steps to Reproduce:** Describe how to reproduce the issue.
4. **Expected Behavior:** Explain what should have happened.
5. **Actual Behavior:** Explain what happened instead.
6. **Environment:** Include relevant operating system, software version, or other details.

For a feature request, describe the problem the feature would solve and explain why it would be useful.

GitHub Issues supports Markdown, so you can use headings, lists, links, code blocks, and task lists to improve readability.

> **Tip:** Be respectful, professional, and specific. A clear issue saves time for everyone involved.

Branches

A branch is an independent line of development within a Git repository.

Branches allow developers to work on new features, fix bugs, or experiment with changes without immediately modifying the main development branch.

The main branch is commonly named `main`, although repositories may use other names.

Why Use Branches?

Imagine that a team is developing a website. One developer is building a login system, while another is fixing a navigation bug.

If both developers work independently on separate branches, they can develop and test their changes without constantly interfering with each other's work.

A typical workflow might look like this:

```
main
 ├── feature/login
 └── fix/navigation
```

Each branch can contain its own commits. Once the changes are reviewed and approved, they can be merged into the target branch.

Creating a Branch

You can create a branch using GitHub Desktop or the command line. A branch is not a separate copy of the entire project in the same sense as a fork. It is a separate line of development within the same repository, sharing the repository's underlying Git history.

Pull Requests

A Pull Request (PR) is a request to review and integrate changes from one branch into another.

For example, after implementing a feature on `feature/login`, you can open a pull request to propose merging it into `main`.

A pull request allows team members to:

- Review code changes.
- Leave comments and suggestions.
- Discuss the proposed implementation.
- Run automated tests and checks.
- Request changes before integration.
- Approve the changes when they are ready.

Creating a Pull Request

1. Create a branch and make your changes.
2. Commit your changes.
3. Push the branch to GitHub.
4. Open the repository on GitHub.
5. Select **Compare & pull request**, when available, or open the **Pull requests** tab and create a new pull request.
6. Choose the source branch and the target branch.
7. Add a descriptive title and explain the changes.

8. Create the pull request and request a review if needed.

> **Important:** Opening a pull request does not automatically merge the changes. The changes must be merged separately by someone with the necessary permissions.

Pull requests are also useful in open-source projects because contributors can propose changes without needing direct write access to the original repository.

Merge

Merging combines changes from one branch into another.

For example, after a pull request has been reviewed and approved, a maintainer may merge the feature branch into `main`.

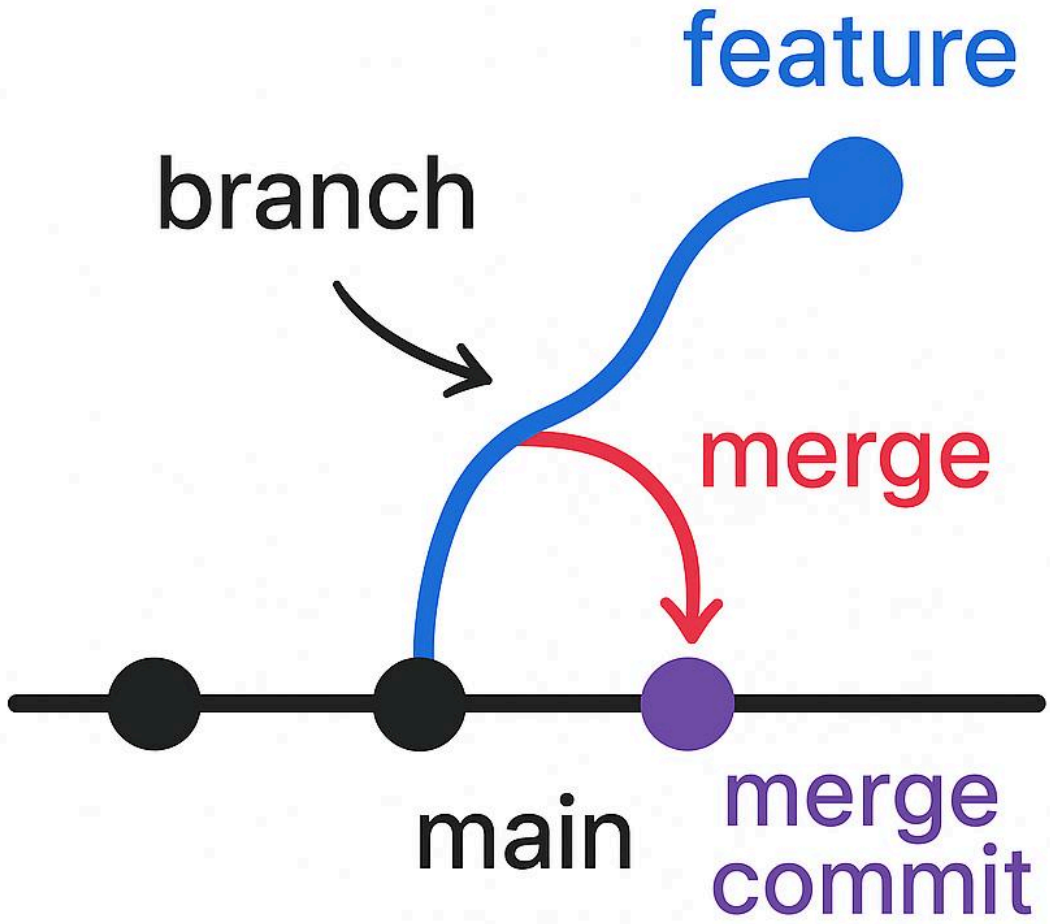
```
Before merging:

main:             A --- B --- C
feature/login:             \--- D --- E

After merging:

main:             A --- B --- C --- M
                                \     /
feature/login:                   D --- E
```

This is a simplified illustration of a merge commit. Git can also perform fast-forward merges or use other merge strategies depending on the repository's history and configuration.



Common Merge Options on GitHub

Create a Merge Commit

Combines the branches and creates a merge commit that preserves the branching history.

Squash and Merge

Combines the pull request's commits into a single commit on the target branch.

This can keep the main branch history more compact.

Rebase and Merge

Replays the pull request's commits onto the target branch without creating a merge commit.

This can produce a linear history, although the resulting commit IDs may differ from the original ones.

The appropriate option depends on the project's workflow and contribution guidelines.

Fork

A fork is a copy of a repository under another GitHub account or organization.

Forking is especially useful when contributing to open-source projects where you do not have permission to push directly to the original repository.

How Forking Works

Imagine that you want to contribute to an open-source project maintained by someone else.

1. Open the original repository on GitHub.
2. Click **Fork**.
3. Choose the destination account or organization.
4. Clone your fork to your computer.
5. Create a branch and make your changes.
6. Commit and push your changes to your fork.
7. Open a pull request proposing that your changes be merged into the original repository.

The workflow can be illustrated as follows:

```
Original Repository
        |
        | Fork
        v
Your Fork
        |
        | Clone
        v
Your Local Computer
        |
        | Edit, Commit, Push
        v
Your Fork on GitHub
        |
        | Pull Request
```

```
        v
Original Repository
```

Branch vs. Fork

Feature	Branch	Fork
Location	Inside the same repository	A separate repository under another account or organization
Typical use	Team development and feature work	Contributing to projects you do not control
Direct repository access required	Usually requires suitable access to push changes	You can generally fork a public repository without write access
Independent repository permissions	No	Yes
Pull requests	Can be opened between branches	Can be opened from your fork to the original repository

A fork is a separate repository, but it is not necessarily a permanent or automatically synchronized copy. You may need to fetch changes from the original repository to keep your fork up to date.

> **Important:** You do not need to be a member of a project to fork its public repository. However, permissions and repository policies may affect the available workflow.

Conclusion

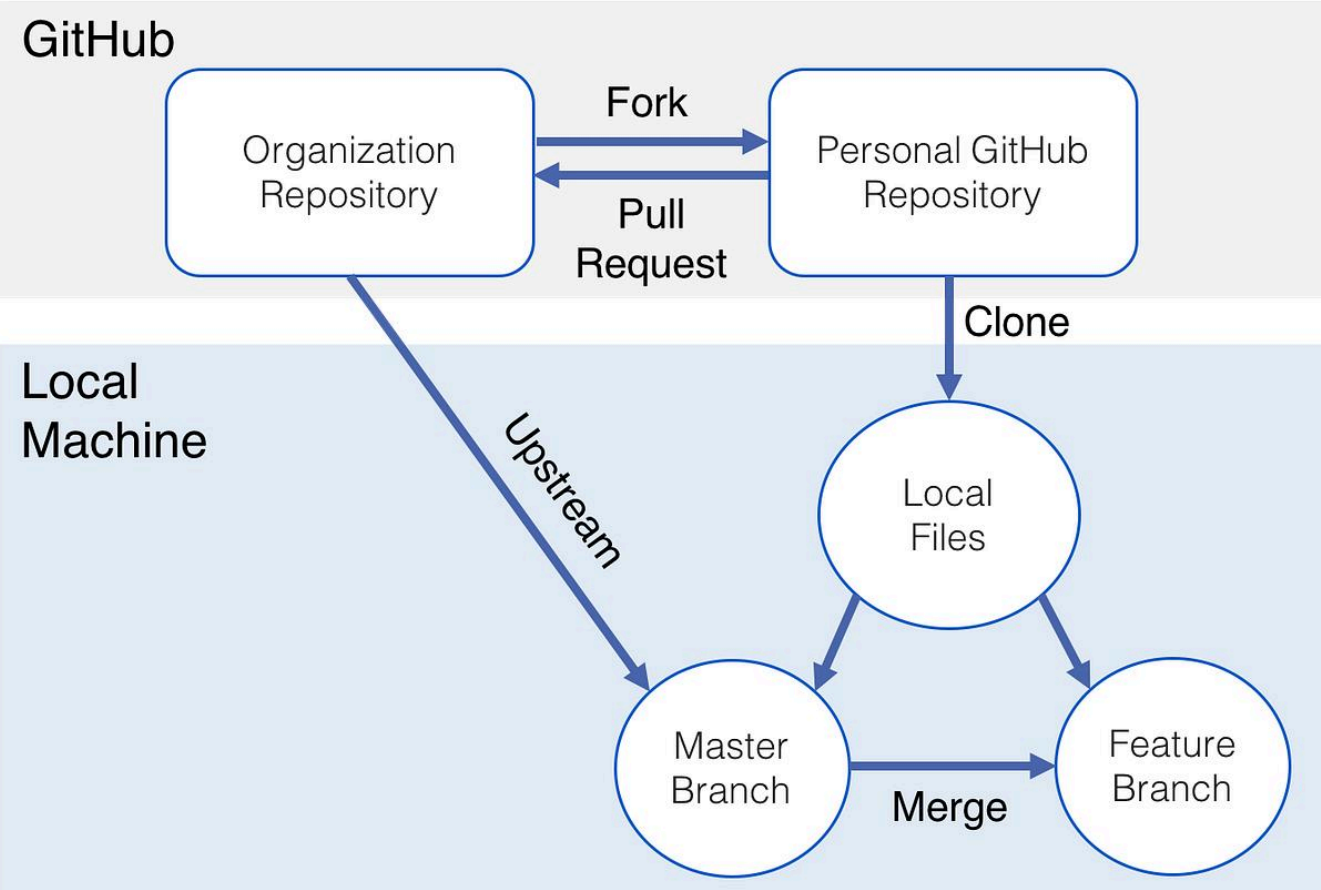
GitHub is an essential platform for modern software development. Learning how to use repositories, Markdown, GitHub Desktop, branches, pull requests, and issues will help you manage projects and collaborate with other developers more effectively.

To improve your GitHub profile:

- Keep your repositories organized.
- Write clear and helpful README files.

- Choose suitable licenses for your projects.
- Use `.gitignore` to avoid committing unnecessary files and local secrets.
- Build a professional profile README.
- Publish your portfolio using GitHub Pages.
- Contribute to open-source projects.
- Use branches and pull requests to develop and review changes.

The best way to learn GitHub is to use it. Start with a small project, publish it, improve it over time, and gradually contribute to projects maintained by other developers.



Useful Resources

- GitHub
- Git Documentation
- GitHub Docs
- GitHub Skills
- GitHub Desktop
- GitHub Pages
- Choose a License
- GitHub Flavored Markdown Specification
- Git Ignore Templates